

```

1 import components.map.Map;
2 import components.map.Map.Pair;
3 import components.map.Map1L;
4 import components.sequence.Sequence;
5 import components.sequence.Sequence1L;
6 import components.set.Set;
7 import components.set.Set1L;
8 import components.simplereader.SimpleReader;
9 import components.simplereader.SimpleReader1L;
10 import components.simplewriter.SimpleWriter;
11 import components.simplewriter.SimpleWriter1L;
12
13 /**
14  * Write a HTML page and make a glossary.
15  *
16  * @author Eric Zhou
17  *
18  */
19 public final class Glossary {
20
21     /**
22      * No argument constructor--private to prevent instantiation.
23      */
24     private Glossary() {
25     }
26
27     /**
28      * get all the terms and definition.
29      *
30      * @param inputFile
31      *      the file inputed by the user
32      * @return the final map
33      *
34      */
35     public static Map<String, String> readGlossary(String inputFile) {
36         Map<String, String> m = new Map1L<>();
37         SimpleReader file = new SimpleReader1L(inputFile);
38
39         while (!file.atEOS()) { //check whether at the end of the file
40             String term = file.nextLine();
41
42             String definition = "";
43             String line = "";
44             do {
45                 line = file.nextLine();
46                 definition = definition.concat(" ").concat(line);
47             } while (!line.equals("") && !file.atEOS());
48             //check next line is definition or the end
49
50             m.add(term, definition.trim());
51
52         }
53
54         file.close();
55         return m;
56     }
57
58     /**
59      * sort the terms by alphabetical order.
60      *
61      * @param seq
62      *      the sequence needed to be sorted
63      */
64     public static void sort(Sequence<String> seq) {
65         for (int i = 0; i < seq.length(); i++) {

```

```

66     for (int j = i + 1; j < seq.length(); j++) {
67         if (seq.entry(j).compareTo(seq.entry(i)) < 0) { //compare the element
68             String temp = seq.remove(j);
69             seq.add(i, temp);
70         }
71     }
72 }
73 }
74
75 /**
76  *
77  * @param terms
78  *     all the terms needed to be added
79  * @param folder
80  *     the aim folder provided by the user
81  */
82 public static void writeIndexPage(Sequence<String> terms, String folder) {
83     SimpleWriter out = new SimpleWriter1L(folder + "/index.html");
84
85     out.println("<html><head><title>Glossary</title></head><body>");
86     out.println("<h2>Glossary Index</h2>");
87     out.println("<ul>");
88     for (int i = 0; i < terms.length(); i++) { //output all the terms
89         String term = terms.entry(i);
90         out.println("<li><a href=\"\" + term + ".html\">\" + term + "</a></li>");
91     }
92     out.println("</ul>");
93     out.println("</body></html>");
94
95     out.close();
96 }
97
98 /**
99  *
100  * @param term
101  *     the specific term
102  * @param def
103  *     the definition of the term
104  * @param terms
105  *     the sequence of the terms with definition
106  * @param folder
107  *     the output folder
108  */
109 public static void writeTermPage(String term, String def, Sequence<String> terms,
110     String folder) {
111     SimpleWriter out = new SimpleWriter1L(folder + "/" + term + ".html");
112
113     out.println("<html><head><title>\" + term + "</title></head><body>");
114     out.println(
115         "<h2><b><i><span style=\"color:red\">\" + term + "</span></i></b></h2>");
116     out.println("<p>\" + addLinks(def, terms) + "</p>");
117     out.println("<p><a href=\"index.html\">Back to index</a></p>");
118     out.println("</body></html>");
119
120     out.close();
121 }
122
123 /**
124  *
125  * @param def
126  *     the definition of term
127  * @param terms
128  *     the sequence of terms
129  * @return the term with link
130  */

```

```

1316 public static String addLinks(String def, Sequence<String> terms) {
132     String result = "";
133     final String separatorChars = " \\t\\n\\r,.;!()?";
134     Set<Character> separators = new Set1L<>();
135     for (int i = 0; i < separatorChars.length(); i++) {
136         separators.add(separatorChars.charAt(i));
137     }
138
139     int position = 0;
140     while (position < def.length()) {
141         String candi = nextWordOrSeparator(def, position, separators);
142
143         boolean linked = false;
144         int i = 0;
145         while (i < terms.length()) {
146             String t = terms.entry(i);
147             if (!linked && candi.equals(t)) { //check whether need to add link
148                 result += "<a href=\"" + t + ".html\"><b><i>" + t + "</i></b></a>";
149                 linked = true;
150             }
151             i++;
152         }
153
154         if (!linked) {
155             result = result.concat(candi);
156         }
157
158         position += candi.length();
159     }
160
161     return result;
162 }
163
164 /**
165  * Returns the first "word" (maximal length string of characters not in
166  * {@code separators}) or "separator string" (maximal length string of
167  * characters in {@code separators}) in the given {@code text} starting at
168  * the given {@code position}.
169  *
170  * @param text
171  *     the {@code String} from which to get the word or separator
172  * @param position
173  *     the starting index
174  * @param separators
175  *     the {@code Set} of separator characters
176  * @return the first word or separator string found in {@code text} starting
177  *     at index {@code position}
178  * @requires 0 <= position < |text|
179  * @ensures <pre>
180  * nextWordOrSeparator =
181  * text[position, position + |nextWordOrSeparator|) and
182  * if entries(text[position, position + 1)) intersection separators = {}
183  * then
184  *     entries(nextWordOrSeparator) intersection separators = {} and
185  *     (position + |nextWordOrSeparator| = |text| or
186  *     entries(text[position, position + |nextWordOrSeparator| + 1))
187  *     intersection separators != {})
188  * else
189  *     entries(nextWordOrSeparator) is subset of separators and
190  *     (position + |nextWordOrSeparator| = |text| or
191  *     entries(text[position, position + |nextWordOrSeparator| + 1))
192  *     is not subset of separators)
193  * </pre>
194  */
195 public static String nextWordOrSeparator(String text, int position,

```

```

196     Set<Character> separators) {
197     String value = "";
198     int index = position;
199     if (separators.contains(text.charAt(position))) { //check the value
200         while (index < text.length() && separators.contains(text.charAt(index))) {
201             index++;
202         }
203     } else { //if the value is not separators
204         while (index < text.length() && !separators.contains(text.charAt(index))) {
205             index++;
206         }
207     }
208     value = text.substring(position, index);
209     return value;
210 }
211
212 /**
213  * Main method.
214  *
215  * @param args
216  *         the command line arguments
217  */
218 public static void main(String[] args) {
219     SimpleReader in = new SimpleReader1L();
220     SimpleWriter out = new SimpleWriter1L();
221
222     out.print("Enter input file name: "); //ask for the file
223     String inputFile = in.nextLine();
224
225     out.print("Enter output folder name: ");
226     String outputFolder = in.nextLine();
227
228     Map<String, String> glossary = readGlossary(inputFile);
229     Sequence<String> terms = new Sequence1L<>();
230     Map<String, String> newGlossary = new Map1L<>();
231
232     while (glossary.size() > 0) {
233         Pair<String, String> pair = glossary.removeAny();
234         String term = pair.key();
235         String def = pair.value();
236         terms.add(terms.length(), term);
237         newGlossary.add(term, def);
238     }
239     glossary.transferFrom(newGlossary);
240     sort(terms); //sort the terms
241
242     writeIndexPage(terms, outputFolder);
243     for (int i = 0; i < terms.length(); i++) {
244         String term = terms.entry(i); //add the definition
245         String def = glossary.value(term);
246         writeTermPage(term, def, terms, outputFolder);
247     }
248
249     in.close();
250     out.close();
251 }
252
253 }
254

```